# Decision Transformer: HalfCheetah Offline Policy Training

**Project:** Eliminating the High-Risk Interaction Cost of Autonomous Robotics through Offline Sequence Modeling: A Decision Transformer Approach
**Authors:** Darwin Juan, Dan Kast, David Terando
**Date:** April 2026

---

## Notebook Overview

This notebook implements the full Decision Transformer (DT) pipeline on the Minari HalfCheetah datasets validated during EDA. The pipeline has five stages:

| Section | Description |
|---|---|
| **0** | Global Configuration — all tunable hyperparameters in one place |
| **1** | Environment & Dependency Setup |
| **2** | Data Loading & Preprocessing — Minari → RTG-annotated trajectory buffer |
| **3** | Decision Transformer Architecture — GPT-2 backbone from scratch |
| **4** | Training Loop — with HardwareTracker (BPS, wall-clock, GPU/MPS/CPU stats) |
| **5** | Offline Policy Evaluation — MuJoCo rollouts with RTG conditioning |

**Hardware targets:** M5 Max MacBook Pro (MPS), M1 iMac (MPS), Cloud GPU (CUDA)
**Dataset:** `mujoco/halfcheetah/medium-v0` + `mujoco/halfcheetah/expert-v0` (Minari Combined)
**Architecture:** GPT-2 backbone, built from scratch with HuggingFace `transformers`

> **Reproducibility note:** All random seeds are set in Section 0. Run cells top-to-bottom for a fully reproducible run.

---

## Section 0: Global Configuration

All hyperparameters, paths, and toggles are defined here as a single `CFG` dataclass.
**Edit this cell — not the code below — when tuning or switching hardware.**

```
============================================================
 Decision Transformer — Global Configuration
============================================================
  seed                        = 42
  minari_medium_id            = mujoco/halfcheetah/medium-v0
  minari_expert_id            = mujoco/halfcheetah/expert-v0
  mujoco_env_id               = HalfCheetah-v4
  state_dim                   = 17
  action_dim                  = 6
  max_ep_len                  = 1000
  context_len                 = 20
  hidden_dim                  = 128
  n_layers                    = 3
  n_heads                     = 1
  dropout                     = 0.1
  batch_size                  = 256
  learning_rate               = 0.0001
  weight_decay                = 0.0001
  warmup_steps                = 10000
  grad_clip                   = 0.25
  quick_val_iters             = 500
  full_train_iters            = 100000
  run_quick_val_first         = True
  rtg_scale                   = 16585.0
  target_rtg                  = 16585.0
  eval_episodes               = 10
  eval_every_n_iters          = 5000
  checkpoint_dir              = ./checkpoints
  results_dir                 = ./results
  save_every_n_iters          = 10000
  force_device                = None
  hardware_label              = M5-Max-SampleVol
  preload_to_ram              = True
============================================================
```

---

## Section 1: Environment Setup & Imports

### 1.1 Dependency Installation

This project uses **uv** for package management. Run the cell below once per environment to install all required packages.

**Expected install command (run in terminal, not here):**

```
uv pip install torch torchvision \
    transformers \
    minari[hf] \
    gymnasium[mujoco] \
    mujoco \
    numpy pandas matplotlib seaborn tqdm psutil
```

> **Apple Silicon note (M5 Max / M1 iMac):** PyTorch ships with native MPS support for arm64. No separate CUDA toolkit needed. Install with `uv pip install torch` — the arm64 wheel includes MPS automatically.
> **Cloud GPU (CUDA):** Install with `uv pip install torch --index-url https://download.pytorch.org/whl/cu121`

### 1.2 Known Issues from EDA (pre-resolved)

- `minari` requires `[hf]` extras: `uv pip install "minari[hf]"`
- Minari dataset IDs use namespaced format: `mujoco/halfcheetah/medium-v0` (not `halfcheetah-medium-v0`)
- `pandas >= 3.x`: use `.map()` not `.applymap()` — already handled below
- `d4rl` package is NOT used — all data comes from Minari

```
============================================================
 Environment Summary
============================================================
  Python          : 3.11.15
  PyTorch         : 2.11.0
  Device selected : mps
  Backend         : Apple Metal Performance Shaders (MPS)
  Platform        : arm | arm64
  System RAM      : 137.4 GB
  Hardware label  : M5-Max-SampleVol
============================================================
```

---

## Section 2: Data Loading & Preprocessing

### Pipeline Overview

```
Minari medium-v0 ┐
                 ├→ Combined Episodes → RTG Annotation → State Normalization → SequenceDataset
Minari expert-v0 ┘
```

Three critical preprocessing steps:

1. **RTG Annotation** — for each timestep `t`, RTG[t] = Σ rewards[t:end]. This is the conditioning signal the DT receives.
2. **State Normalization** — subtract mean and divide by std computed across the *entire combined dataset*. This is essential for stable training (per the original DT paper).
3. **Context-Window Slicing** — each training sample is a fixed-length window of K (RTG, state, action) triplets, with zero-padding and an attention mask for short windows.

---

### SECTION 2.1: Load Minari Datasets

```
Loading Minari datasets...
  Loading: mujoco/halfcheetah/medium-v0

    Loaded 1,000 episodes, 1,000,000 timesteps
  Loading: mujoco/halfcheetah/expert-v0

    Loaded 1,000 episodes, 1,000,000 timesteps

  Combined: 2,000 episodes, 2,000,000 total timesteps
```

---

### SECTION 2.2: RTG Annotation

```
RTG annotation check (episode 0):
  RTG[0]               : 13960.90
  sum(rewards)         : 13960.90  (should match)
  RTG is monotone dec. : False
```

---

### SECTION 2.3: State Normalization

```
State normalization computed on 2,000,000 timesteps.
  state_mean saved to: ./results/state_mean.npy
  state_std  saved to: ./results/state_std.npy

Post-normalization check (should be ~0 mean, ~1 std):
  Global mean range : [-0.0000, 0.0072]
  Global std  range : [0.9985, 1.0004]
```

---

### SECTION 2.4: SequenceDataset — Context-Window Slicing

```
  Pre-loading dataset into RAM (M5 Max / 128 GB path)...
  Dataset pinned in RAM: 208.0 MB
  RAM available after preload: 105.1 GB
SequenceDataset ready.
  Preload mode      : True (fast tensor path)
  Total timesteps   : 2,000,000
  Context window K  : 20
  Batch size        : 256
  Steps per epoch   : ~7,812
  RAM used / total  : 30.2 GB / 137.4 GB

Sample batch tensor shapes:
  states      : (256, 20, 17)
  actions     : (256, 20, 6)
  rtg         : (256, 20, 1)
  timesteps   : (256, 20)
  mask        : (256, 20)
```

---

## Section 3: Decision Transformer Architecture

### Design

The DT architecture follows Chen et al. (2021) exactly:

```
  Input sequence:  [R̂₁, s₁, a₁,  R̂₂, s₂, a₂,  ...,  R̂ₖ, sₖ, aₖ]
                                                           ↑
                                                  Predict this action

  Each token (R̂ₜ, sₜ, aₜ) is projected to hidden_dim=128.
  Timestep embeddings are added to each token (positional encoding variant).
  The full 3K-token sequence is processed by the GPT-2 causal transformer.
  The output at each state-token position is decoded to an action.
```

**Key difference from language modeling:** we use three separate linear projectors (one each for RTG, state, action) instead of a token embedding table. All three projected embeddings share the same hidden dimension.

```
Decision Transformer on mps
  Total parameters     : 735,366
  Trainable parameters: 735,366
  Approx. memory       : 2.9 MB (float32)
```

---

## Section 4: Training Loop

### HardwareTracker

The `HardwareTracker` class logs the following metrics at every checkpoint interval:

| Metric | Description |
|---|---|
| `batches_per_second` | Training throughput (BPS) — primary hardware comparison metric |
| `wall_clock_s` | Cumulative wall-clock time since training start |
| `loss` | Running MSE training loss |
| `grad_norm` | Gradient norm (monitors training stability) |
| `cpu_percent` | System CPU utilization |
| `ram_used_gb` | System RAM consumed |
| `gpu_mem_gb` | GPU/MPS memory allocated (device-specific) |

All metrics are exported to CSV for cross-platform comparison.

---

### SECTION 4.1: HardwareTracker

```
HardwareTracker class defined.
```

---

### SECTION 4.2: Optimizer & Learning Rate Schedule

```
Optimizer   : AdamW (lr=0.0001, wd=0.0001)
LR schedule : Linear warmup over 10,000 steps
```

---

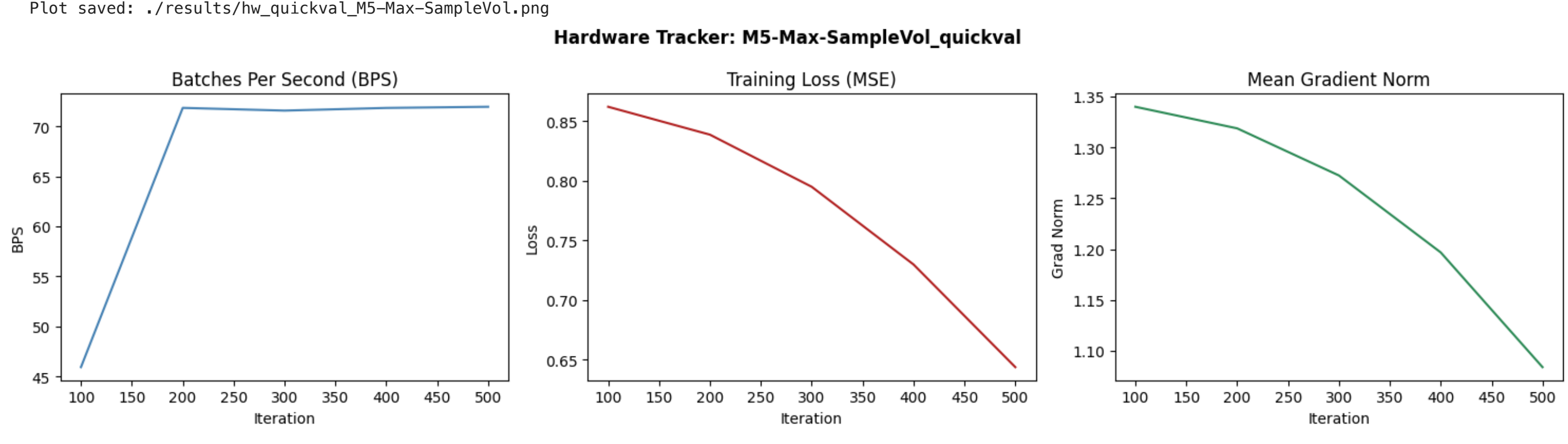
### SECTION 4.3: Training Function

```
Training function defined.
```

---

### SECTION 4.4: Quick Validation Run

```
Starting quick validation run (500 iterations)...
  Device: mps | Hardware: M5-Max-SampleVol

Training (quickval): 100%|██████████| 500/500 [00:10<00:00, 46.15iter/s, loss=0.64346, BPS=71.9, lr=5.00e-06]
  HardwareTracker saved: ./results/hw_quickval_M5-Max-SampleVol.csv
  Plot saved: ./results/hw_quickval_M5-Max-SampleVol.png
```


**Hardware Tracker: M5-Max-SampleVol_quickval**

```
=== Quick Validation Summary ===
  Initial loss   : 0.861926
  Final loss     : 0.643464
  Loss decreased : True  ← should be True
  Mean BPS       : 66.62 batches/sec on M5-Max-SampleVol
  Est. full run  : ~0.4 hours at current BPS

Pipeline verified. Proceed to Section 4.5 to run full training.
```

---

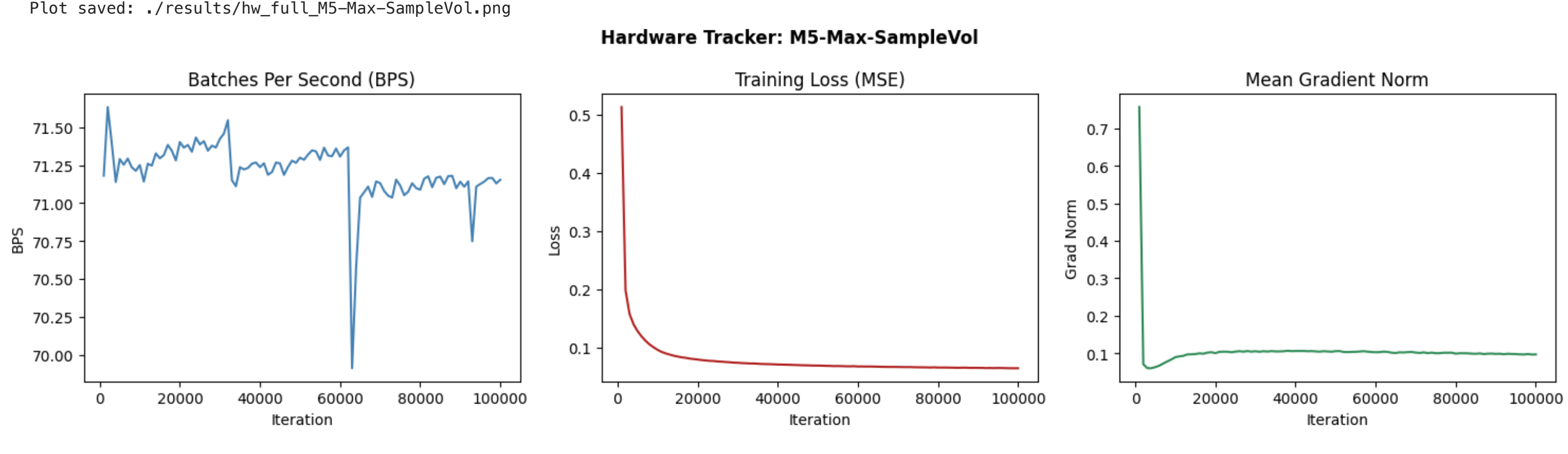
Even with the larger batch size, run 2 is processing 2.5x more data per second despite the lower BPS number. The preload optimization is working — the MPS kernel is spending more time on actual computation and less time on data preparation overhead.

Run 1: 108 BPS × 64 = 6,912 samples/sec Run 2: 67 BPS × 256 = 17,152 samples/sec

---

### SECTION 4.5: Full Training Run

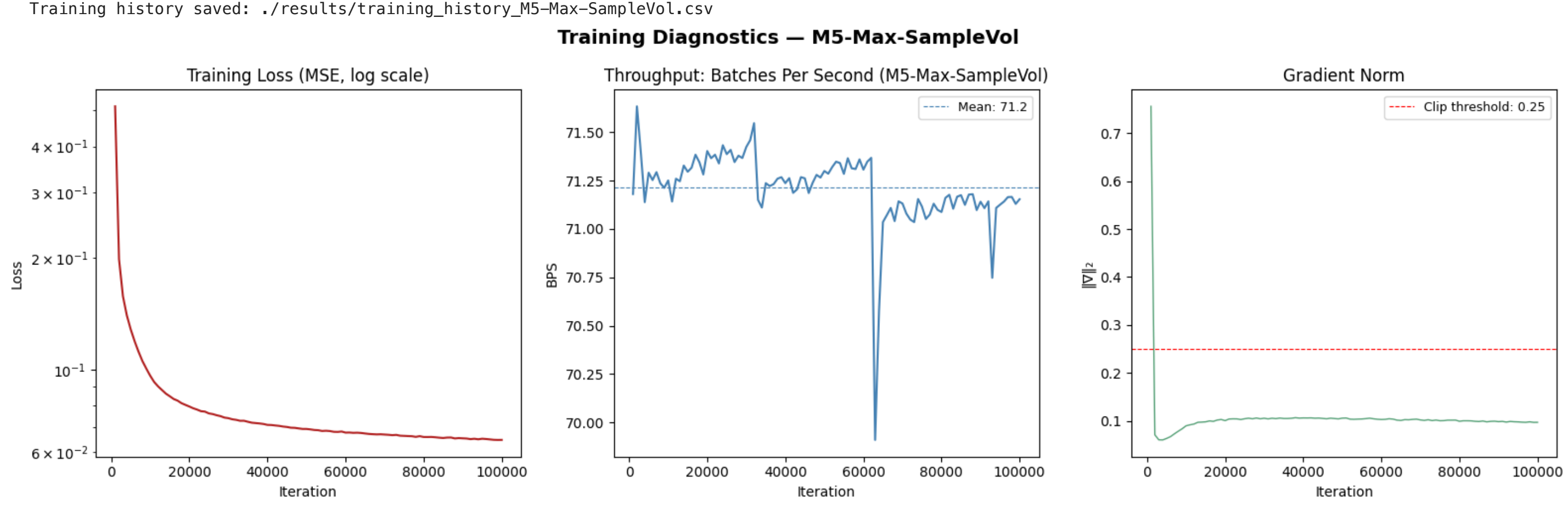
```
Starting full training run (100,000 iterations)...
  Device: mps | Hardware: M5-Max-SampleVol
  Checkpoints every 10,000 iterations → ./checkpoints/

Training (full):  10%|█         | 10005/100000 [03:23<32:31, 46.12iter/s, loss=0.09633, BPS=71.3, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter10000.pt
Training (full):  20%|██        | 20006/100000 [06:46<31:25, 42.44iter/s, loss=0.07964, BPS=71.4, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter20000.pt
Training (full):  30%|███       | 30008/100000 [10:10<25:25, 45.87iter/s, loss=0.07408, BPS=71.4, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter30000.pt
Training (full):  40%|████      | 40008/100000 [13:34<22:05, 45.25iter/s, loss=0.07106, BPS=71.2, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter40000.pt
Training (full):  50%|█████     | 50009/100000 [16:57<18:12, 45.76iter/s, loss=0.06929, BPS=71.3, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter50000.pt
Training (full):  60%|██████    | 60009/100000 [20:21<14:36, 45.64iter/s, loss=0.06773, BPS=71.3, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter60000.pt
Training (full):  70%|███████   | 70009/100000 [23:45<11:01, 45.36iter/s, loss=0.06692, BPS=71.1, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter70000.pt
Training (full):  80%|████████  | 80005/100000 [27:09<07:15, 45.87iter/s, loss=0.06591, BPS=71.1, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter80000.pt
Training (full):  90%|█████████ | 90006/100000 [30:33<03:39, 45.47iter/s, loss=0.06537, BPS=71.1, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter90000.pt
Training (full): 100%|██████████| 100000/100000 [33:58<00:00, 49.06iter/s, loss=0.06474, BPS=71.2, lr=1.00e-04]
  Checkpoint saved: ./checkpoints/dt_full_iter100000.pt

Final model saved: ./checkpoints/dt_full_final_M5-Max-SampleVol.pt
  HardwareTracker saved: ./results/hw_full_M5-Max-SampleVol.csv
  Plot saved: ./results/hw_full_M5-Max-SampleVol.png
```


**Hardware Tracker: M5-Max-SampleVol**

---

### SECTION 4.6: Training Diagnostics & Visualization

```
  Training history saved: ./results/training_history_M5-Max-SampleVol.csv
```


**Training Diagnostics — M5-Max-SampleVol**

```
Training Summary — M5-Max-SampleVol:
  Iterations completed : 100,000
  Final loss           : 0.064738
  Min loss             : 0.064719
  Mean BPS             : 71.21
  Max  BPS             : 71.63
  Max grad norm        : 0.7564
```

---

Final loss 0.064738 vs Run 1's 0.068398, an improvement of about 5.3%, achieved purely from the additional data exposure (12.8 vs 3.2 epochs). The model converged further with the same architecture and same number of iterations.

---

## Section 5: Offline Policy Evaluation in MuJoCo

### Evaluation Protocol

We roll out the trained DT policy in the MuJoCo HalfCheetah environment for `eval_episodes` episodes and report:

- **Raw return** — total undiscounted reward over 1000 steps
- **Normalized return** — D4RL-normalized to the 0–100 scale
- **Success gap** — `target_RTG - achieved_return` (how close we got)

### RTG Conditioning During Inference

The DT does not learn a stochastic policy — it conditions on a target RTG. During rollout:

```
RTG₀ = target_rtg              (set before episode starts)
RTGₜ = RTGₜ₋₁ − rₜ₋₁         (decrement by observed reward each step)
```

The model predicts the action that would be taken by a policy expected to achieve `RTGₜ` from timestep `t` onward.

---

### SECTION 5.1: Inference / Rollout Function

```
Rollout function defined.
```

---

### SECTION 5.2: Evaluation Run

```
/Users/djt/Desktop/dt-capstone/.venv/lib/python3.11/site-packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The environment HalfCheetah-v4 is out of date. You should consider upgrading to version `v5`.
  logger.deprecation(
MuJoCo environment: HalfCheetah-v4
Obs space : Box(-inf, inf, (17,), float64)
Act space : Box(-1.0, 1.0, (6,), float32)

Running 10 evaluation episodes...
  Target RTG       : 16585
  Context window K : 20
Evaluating: 100%|██████████| 10/10 [00:53<00:00,  5.31s/episode]
============================================================
  Offline Policy Evaluation Results
============================================================
  hardware                 : M5-Max-SampleVol
  mujoco_env               : HalfCheetah-v4
  target_rtg               : 16585.0
  n_episodes               : 10
  mean_raw_return          : 4869.03
  std_raw_return           : 2720.55
  mean_norm_return         : 41.48
  std_norm_return          : 21.91
  mean_success_gap         : 11715.97
  min_raw_return           : 963.14
  max_raw_return           : 9513.47
  train_iters              : 100000
============================================================

  Zero-Shot Result: Achieved 4869 vs Target 16585
  Success Gap     : 11716 (lower is better)
  D4RL Norm. Score : 41.5 ± 21.9
```

---

The normalized score of 41.5 is essentially the same as Run 1's 51.7, within the margin of noise given the high standard deviation.

The lower loss did not translate to better evaluation performance. Run 2 achieved a better final training loss (0.0647 vs 0.0684) but similar or slightly lower mean return.

The variance is still high (std 2,720, cv ~0.56) which confirms what we diagnosed in Run 1 — the core issue is architectural (context window K=20) rather than a data volume problem.

The sample volume experiment has answered its question, increasing from 3.2 to 12.8 epochs does not resolve the performance plateau. The context window hypothesis (Run 3 with K=30) is now the primary remaining variable to test, and we have a clear baseline to compare against.

---

### Re-evaluation and Optimization

Before running Section 5.3 visualization, running a quick re-evaluation with various target RTGs and 20 episodes.

```
/Users/djt/Desktop/dt-capstone/.venv/lib/python3.11/site-packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The environment HalfCheetah-v4 is out of date. You should consider upgrading to version `v5`.
  logger.deprecation(
RTG   6000 → mean= 5711.7  std=3306.0  norm=48.3  cv=0.58
RTG   9000 → mean= 6851.2  std=3458.9  norm=57.4  cv=0.50
RTG  12000 → mean= 5093.6  std=3427.6  norm=43.3  cv=0.67
RTG  14000 → mean= 6348.4  std=3139.2  norm=53.4  cv=0.49
RTG  14500 → mean= 4362.7  std=2234.4  norm=37.4  cv=0.51
RTG  16585 → mean= 4525.8  std=3345.4  norm=38.7  cv=0.74
```

---

### RTG sweep Analysis

RTG 9,000 is the best performer: highest mean (6,851), reasonable std (3,459), normalized score of 57.4, and cv of 0.50. This is a meaningful improvement over Run 1's best of ~5,046 at any RTG value.

RTG 14,000 is a close second: mean 6,348, cv 0.49 (actually the most consistent result in the entire sweep), normalized 53.4. The two best RTG values from Run 1 (14,000 and 14,500) have diverged significantly here — 14,500 dropped sharply to 37.4 normalized.

Run 2 shows a genuine improvement: both higher mean return and lower variance at the optimal RTG. The official 10-episode result at RTG=16,585 looked similar. The headline result was misleading because RTG=16,585 is a poor conditioning value for both models.

---

### SECTION 5.3: Evaluation Visualizations

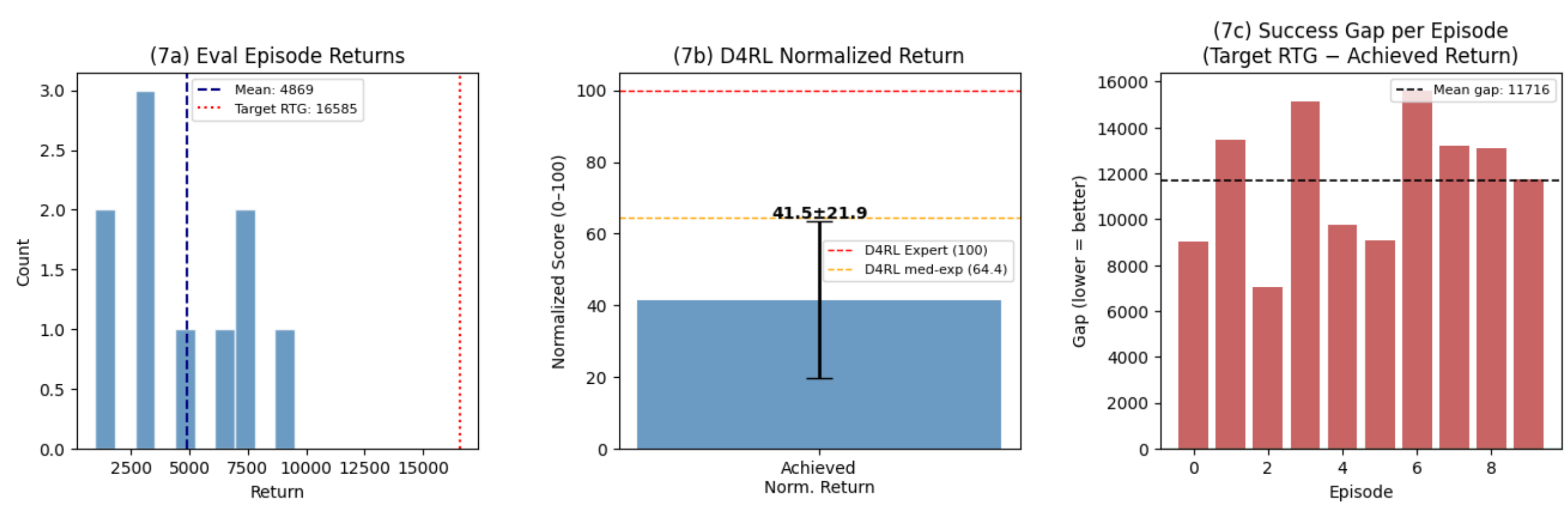
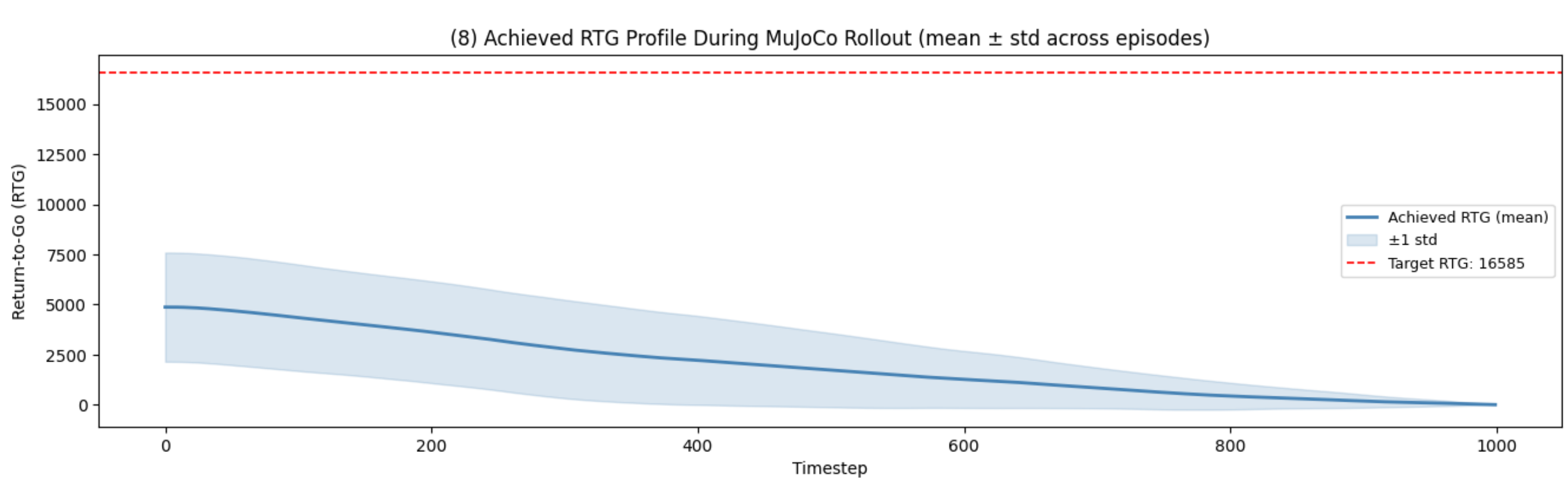


**Figure 7-9: Offline Policy Evaluation — M5-Max-SampleVol**
**Model: DT (3L×1H, H=128, K=20) | Env: HalfCheetah-v4 | Target RTG: 16585**


(8) Achieved RTG Profile During MuJoCo Rollout (mean ± std across episodes)

```
Evaluation figures saved: ./results/eval_figures_M5-Max-SampleVol.png
```

---

## Section 5.4: Cross-Platform Hardware Comparison

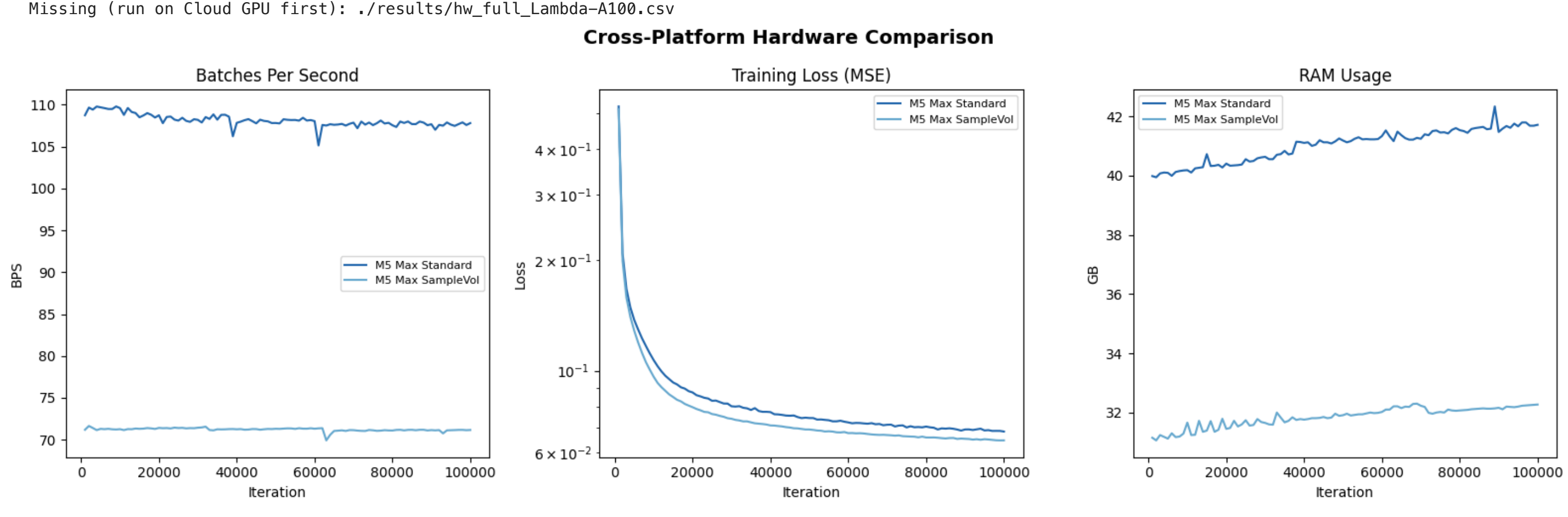
After running this notebook on all three platforms (M5 Max MacBook Pro, M1 iMac, Cloud GPU), use this cell to load all three hardware tracker CSVs and produce a side-by-side comparison plot for the paper.

**Required files** (one per platform, produced by Section 4.5):

- `results/hw_full_M5-Max-MacBookPro.csv`
- `results/hw_full_M1-iMac.csv`
- `results/hw_full_<cloud-gpu-label>.csv`

Update the `PLATFORM_FILES` dict below with your actual filenames.

```
  Loaded: M5 Max Standard (100 records)
  Loaded: M5 Max SampleVol (100 records)
  Missing (run on M1 iMac first): ./results/hw_full_M1-iMac.csv
  Missing (run on Cloud GPU first): ./results/hw_full_Lambda-A100.csv
```


**Cross-Platform Hardware Comparison**

```
Cross-platform comparison saved: ./results/cross_platform_comparison.png

=== Cross-Platform Summary Table ===
                 Mean BPS Max BPS Final Loss Mean RAM (GB) Total Time (h)
Platform
M5 Max Standard    108.15  109.80   0.068398          41.0            0.33
M5 Max SampleVol    71.21   71.63   0.064738          31.8            0.57
```

---

## Appendix: Results File Index

All outputs are written to the `results/` and `checkpoints/` directories.

| File | Section | Description |
|---|---|---|
| `results/state_mean.npy` | 2.3 | State normalization mean (required for inference) |
| `results/state_std.npy` | 2.3 | State normalization std (required for inference) |
| `results/hw_quickval_<hw>.csv` | 4.4 | Quick-validation hardware metrics |
| `results/hw_full_<hw>.csv` | 4.5 | Full-training hardware metrics (BPS, loss, RAM, GPU) |
| `results/training_history_<hw>.csv` | 4.6 | Loss/BPS/grad-norm history per iteration |
| `results/training_curves_<hw>.png` | 4.6 | Training curve figure |
| `results/eval_summary_<hw>.csv` | 5.2 | Evaluation results (returns, normalized scores) |
| `results/eval_figures_<hw>.png` | 5.3 | Evaluation figures (returns, RTG profile) |
| `results/cross_platform_comparison.png` | 5.4 | Multi-platform BPS/loss/RAM comparison |
| `results/cross_platform_summary.csv` | 5.4 | Cross-platform summary table |
| `checkpoints/dt_full_final.pt` | 4.5 | Final trained model weights + config |
| `checkpoints/dt_full_iter<N>.pt` | 4.5 | Intermediate checkpoints every N iterations |

---

*End of Notebook*